

UNIVERSITY OF BIRMINGHAM

School of Computer Science

Data Structures & Algorithms

Sample Exam Questions

Data Structures & Algorithms

1. One can, inefficiently, implement a queue using two stacks, without using any further data structures such as arrays, linked lists etc.
 - (a) Explain the basic approach to implementing a queue this way. [5 marks]
 - (b) Write the pseudocode for the enqueue function. [5 marks]
 - (c) Write the pseudocode for the dequeue function. [5 marks]
 - (d) Analyse, in terms of $O(g(n))$, the complexity of each operation and explain, briefly, your reasoning. [5 marks]

Topics: 02 - Stacks and Queues, 03 - Efficiency and Complexity

2. The handout document for this module (page 37) gives the pseudocode for a recursive version of a function to calculate the size of a binary tree.

```
1 size(t)
2 {
3     if ( isEmpty(t) )
4         return 0
5     else
6         return (1 + size(left(t)) + size(right(t)))
7 }
```

Write a non-recursive pseudocode version using a stack to keep track of the work still to be done. [20 marks]

Topics: 04 - Trees

3. Construct a **minimal** AVL tree of positive integers and of height 5
 - (a) Draw that AVL tree. [5 marks]
 - (b) Identify a value that on insertion into the tree would not trigger any rotation. [5 marks]
 - (c) Identify a value that on insertion into the tree would trigger precisely one rotation. [5 marks]
 - (d) Identify an insertion into that tree that would trigger precisely one double rotation, i.e. a right rotation followed by a left rotation or a left rotation followed by a right rotation. [5 marks]

Topics: 05 - AVL-Trees

4. For each of the following functions to calculate the largest element of an array, work out the formula for the number of operations (assignments, comparisons and arithmetic operations) executed as a function of the size of the array, n , state which complexity class in big O notation the function belongs to and justify your answer with a short explanation. Assume that the sort algorithm used in (c) takes $O(n \log n)$ operations.

```
(a) int largest1(int [] arr) {  
2   int n = arr.length;  
3   int max = 0;  
4   for (int i=0; i<n; i++) {  
5       bool largest = true;  
6       for (int j=0; j<n; j++) {  
7           if (arr[i] < arr[j])  
8               largest = false;  
9       }  
10      if (largest)  
11          max = arr[i];  
12  }  
13  return max;  
14 }
```

[7 marks]

```
(b) int largest2(int [] arr) {  
2   int n = arr.length;  
3   int max = 0;  
4   if (arr.length == 0) {  
5       return 0;  
6   } else {  
7       max = arr[0];  
8       for (int i=0; i<n; i++) {  
9           if (arr[i] > max)  
10              max = arr[i];  
11      }  
12      return max;  
13  }  
14 }
```

[7 marks]

```
(c) int largest3(int [] arr) {  
2   sort(arr);  
3   if (arr.length == 0) {  
4       return 0;  
5   } else {  
6       int last = arr[arr.length - 1];  
7       return last;  
8   }  
9 }
```

[6 marks]

Topics: : 03 - Efficiency and Complexity

5. Consider a binary search tree used to store integers with methods `isEmpty(t)`, `left(t)`, `right(t)` and `root(t)`, to return whether the tree `t`, is empty, the left child tree, the right child tree and the integer value stored at the root respectively.

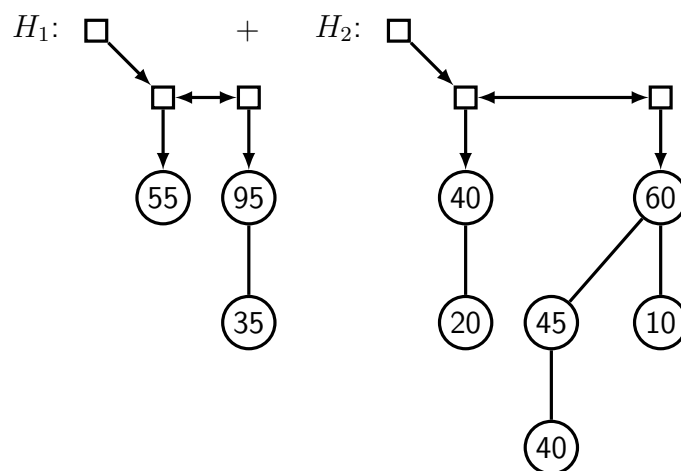
- Write a **recursive** function `sum_rec(tree)` to calculate and return the sum of all integers stored in the tree. **[8 marks]**
- A `Queue` ADT has, for a queue `q`, methods `q.enqueue(val)` and `q.dequeue()` to enqueue a value `val` in the queue and to dequeue and return a value from the queue respectively. It also has a constructor which can be invoked with the command `new Queue()` to create and return a new empty queue.

Write the pseudocode for a **non-recursive** function `sum_nonrec(tree)`, to calculate and return the sum of all integers stored in the tree. **[12 marks]**

Topics: 02 - Stacks and Queues, 04 - Trees

6. (i) Consider a Binary Heap where the highest priority is the highest value. The standard Binary Heap `heapify` method is used to construct this Binary Heap on an Array that contains, in index locations 1 to 10, the integers 1 to 10. Choose and write out an ordering of these integers in the array that **maximises** the number of swaps that the `heapify` method will execute on the array and report the total number of swaps executed. **[10 marks]**

(ii) Draw the result of merging the following two Binomial Heaps:



[10 marks]

Topics: 07 - Priority Queues and Heap Trees